

Managing Console I/O Operations

C++ Streams

- The system gets some input, processes that and then gives the output.
- Here it is important to know that how to give input and how to get output in a desired format.
- In C++ we have stream classes to implement its I/O operations.
- Here the I/O system supplies an interface to the programmer that is independent of actual device being used.
- This Interface is called *STREAM*.
- A C++ **stream** is a flow of data into or out of a program, such as the data written to cout or read from cin.
- For this , we have four different classes:
 - **istream** is a general purpose input stream. cin is an example of an istream.
 - **ostream** is a general purpose output stream. cout is an example of ostream.
 - **ifstream** is an input file stream. It is a special kind of an istream that reads in data from a data file.
 - **ofstream** is an output file stream. It is a special kind of ostream that writes data out to a data file.

Stream

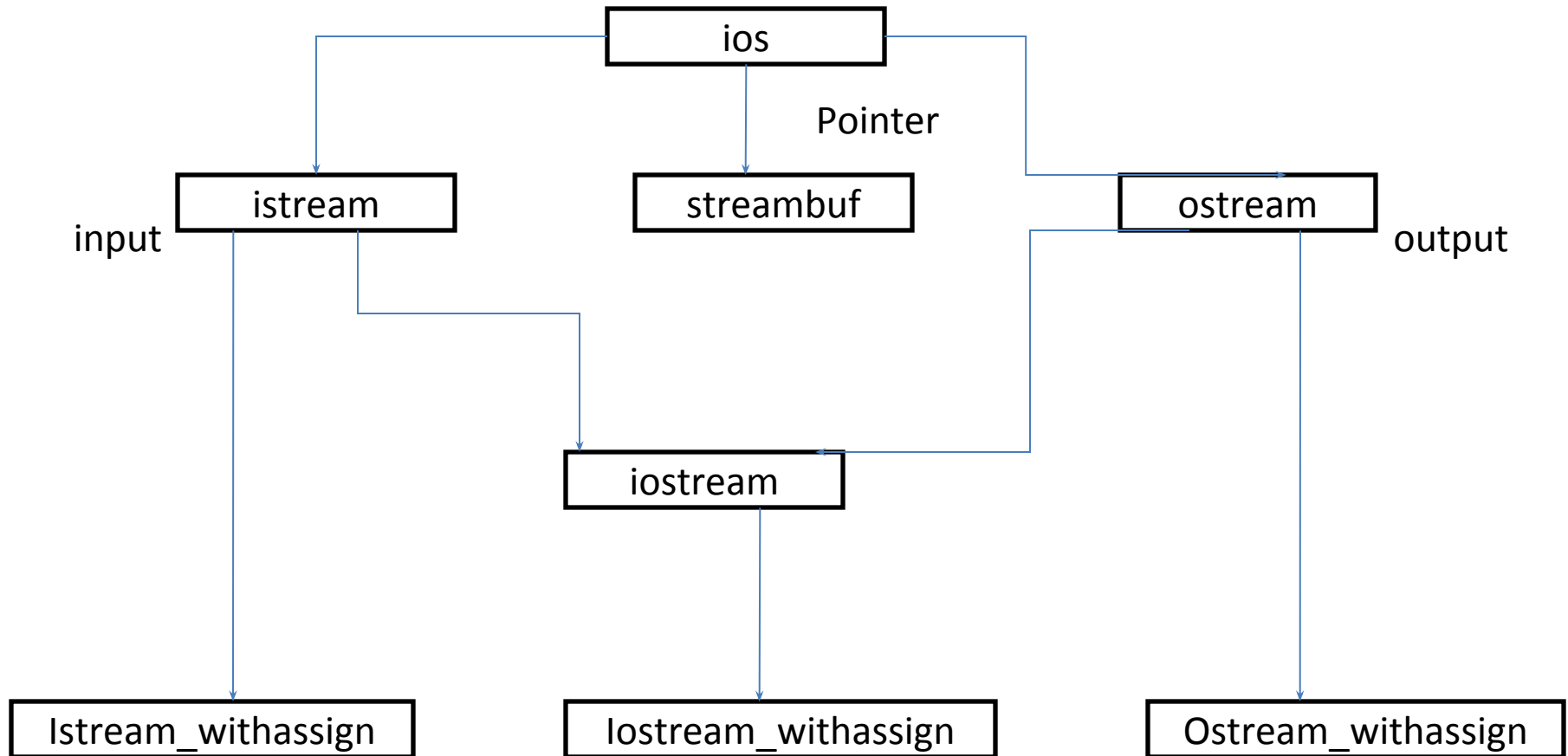
-
- A stream is a sequence of bytes that act as a source for input and as a destination for output.
- The source stream is called Input Stream and destination stream is called Output Stream.
- As the stream acts as an interface between Program and the Input and Output devices hence C++ can handle data independent of the devices used.
- Cin and cout are the streams that are automatically opened when the program is executed. Used for keyboard and the screen.

The >> operator is used in *istream* class. And

The << operator is used in *ostream* class.

Hierarchy of Console Stream Classes

- C++ I/O system contains a hierarchy of classes that are used to define various streams to deal with both the console and the disk files.
- These classes are called *stream classes*.



HIERARCHY OF STREAM CLASSES

Hierarchy of Console Stream Classes

- All the classes in the above diagram are declared in the header file *iostream*.
- ios is the Base class for istream and ostream, which are in turn base classes for iostream class.
- Here ios class is declared as virtual base class so that only one copy of its members are inherited by the iostream.
- The ios provides basic support for formatted and unformatted I/O operations.
- Similarly the istream provides facilities for formatted and unformatted Input and ostream provides facilities for formatted output operations.
- Hence iostream has the features of both istream and ostream.

Unformatted I/O Operations

- Put() and Get() : Get() is defined in istream and Put() is defined in ostream, to handle single character I/O operations.

General Syntax:

cin.get();
cout.put();

```
1  Character I/O with get() and put()
2  #include <iostream.h>
3  using namespace std;
4  int main()
5  {
6      int count=0;
7      char c;
8      cout<<"INPUT TEXT \n";
9      cin.get( c );
10     while ( c !='\n' )
11     { cout.put( c);
12       count++;
13       cin.get( c );
14     }
15     cout<< "\n Number of characters =" ----"\n";
16     return 0;
17 }
18 Input
19   Object oriented programming
20 Output
21   Object oriented programming
22   Number of characters=27
```

Managing Output with Manipulators

- The Header File *iomanip* provides the set of functions called *manipulators* used to manipulate the output formats. Two or more manipulators can also be used in chain as shown:

```
cout<<manip1<<manip2<<manip3<<item;
```

```
cout<<manip1<<item1<<manip2<<item2;
```

This kind of concatenation is used when the data has to be displayed in multiple columns.

Manipulator	Meaning	Equivalent
Setw(int w)	Set the field width to w	Width()
Setprecision(int d)	Set the floating point precision to d	Precision()
Setfill(int c)	Set the fill character to c	Fill()
Setiosflags(long f)	Set the format flag f	Setf()
Resetiosflags(long f)	Clears the flag specified by f	Unsetf()
Endl	Insert new line and flush stream	"\n"

Managing Output with Manipulators

e.g.

```
cout<<setw(10)<<12345;
```

It prints 12345 right justified in the width of ten characters.

The output can be made left justified for the same as:

```
cout<<setw(10)<<setiosflags(ios::left)<<12345;
```

Another Example;

```
cout<<setw(5)<<setprecision(2)<<1.2345
```

```
<<setw(10)<<setprecision(4)<<sqrt(2)
```

```
<<setw(15)<<setiosflags(ios::scientific)<<sqrt(3);
```

All three values will be printed in one line with the field sizes of 5,10 and 15 resp: Here all outputs are controlled by the different sets of format specifications.

Managing Output with Manipulators

Manipulators can also be used with the ios functions in the program.

e.g.

```
cout.setf(ios::showpoint);  
cout.setf(ios::showpos);  
cout<<setprecision(4);  
cout<<setiosflags(ios::scientific);  
cout<<setw(10)<<123.45678;
```

The difference between the manipulators and the member functions is that ios member function return the previous format state which can be used later, but manipulators does not return the previous state

```
cout.precision(2);           //Previous State  
int p = cout.precision(4);   // Current State
```

When executed, p will hold the value of 2(previous state) and the new format state will be 4. we can restore the previous format as :

```
cout.precision(p);           // p=4
```

Manipulator functions

- It facilitate the formatting of input and output streams.
- For this we include the header file i.e

`#include<iomanip.h>`

Manipulators functions specifically designed to be used in conjunction with the insertion (<<) and extraction (>>) operators on stream objects.

dec	uppercase
endl	ws
ends	nouppercase
fixed	oct
flush	resetiosflags
hex	right
internal	scientific
setiosflags	setbase

C++ Manipulator - setprecision

```
#include <iostream>
```

```
#include <iomanip>
```

```
using namespace std;
```

```
int main (void)
```

```
{
```

```
    float a,b,c;
```

```
    a = 5;
```

```
    b = 3;
```

```
    c = a/b;
```

```
    cout << setprecision (1) << c << endl;
```

```
    cout << setprecision (2) << c << endl;
```

```
    cout << setprecision (3) << c << endl;
```

```
    cout << setprecision (4) << c << endl;
```

```
    cout << setprecision (5) << c << endl;
```

```
    cout << setprecision (6) << c << endl;
```

```
    return 0;
```

```
}
```

output:

2

1.7

1.67

1.667

1.6667

1.66667

C++ Manipulator - setw

C++ manipulator setw function stands for **set width**. This manipulator is used to specify the minimum number of character positions on the output field a variable will consume.

```
#include <iostream>
```

```
#include <iomanip>
```

```
using namespace std;
```

```
int main (void)
```

```
{
```

```
    int a,b;
```

```
    a = 200;
```

```
    b = 300;
```

```
    cout << setw (5) << a << setw (5) << b << endl;
```

```
    cout << setw (6) << a << setw (6) << b << endl;
```

```
    cout << setw (7) << a << setw (7) << b << endl;
```

```
    cout << setw (8) << a << setw (8) << b << endl;
```

```
    return 0; }
```

output:

```
200 300
```

```
200 300
```

```
200 300
```

```
200 300
```

C++ manipulator dec function

It is used to set the basefield format flag for the str stream.

When we set basefield to **dec**, then integer values inserted into the stream are expressed in decimal base (i.e. radix 10). For input stream, when this flag is set, then extracted values are also expected to be expressed in decimal base.

```
#include <iostream>

#include <sstream>

using namespace std;

int main()
{
    cout << "The number 42 in octal:  " << oct << 42 << '\n'
         << "The number 42 in decimal: " << dec << 42 << '\n'
         << "The number 42 in hex:    " << hex << 42 << '\n';

    int n;

    istringstream("2A") >> hex >> n;

    cout << dec << "Parsing \"2A\" as hex gives " << n << '\n'; }
```

Output :

```
The number 42 in octal: 52
The number 42 in decimal: 42
The number 42 in hex: 2a
Parsing "2A" as hex gives 42
```

C++ manipulator ends function

It is used to insert a null terminating character on os. The ends manipulator does not take any argument whenever it is invoked. It causes a null character to the output.

It is similar to '\0' or charT() in C++.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    double x, y;
```

```
    cout << "Enter two Floating Values: ";
```

```
    cin >> x ;
```

```
    cin >> y;
```

```
    cout << "The Average of the two Numbers is : " << (x +y)/2.0 << endl;
```

```
    cout<< ends;  // Null Terminate
```

```
    return 0; }
```

Output:

Enter two Floating Values: 100 200

The Average of the two Numbers is : 150

C++ Manipulator setbase

C++ **manipulator setbase** is used to convert the basefield of one numeric value into another base. Possible values are:

dec: decimal base (base = 10)

hex: hexadecimal base (base = 16)

oct: octal base (base = 8)

zero: If base is any other value.

Output:

Example:

6e

```
#include <iostream>    // std::cout, std::endl
```

```
#include <iomanip>      // std::setbase
```

```
using namespace std;
```

```
int main () {
```

```
    cout << setbase(16);
```

```
    cout << 110 << endl;
```

```
    return 0; }
```

C++ Manipulator ws

C++ **manipulator ws** function is used to discard leading whitespace from an input stream.

```
#include <iostream>
```

```
using namespace std;
```

```
int main () {
```

```
    char str1[256], str2 [256];
```

```
    cout << "Please, enter some whitespaces followed by  
    a sentence:\n";
```

```
    cin >> ws;
```

```
    cin.getline (str1,256);
```

```
    cout << "Please, enter some whitespaces followed by a sentence:\n";
```

```
    ws (cin);
```

```
    cin.getline (str2,256);
```

```
    cout << "\nYou wrote:" << endl << str1 << endl << str2 << endl;
```

```
    return 0;
```

```
}
```

Output:

Hello!!

Please, enter some whitespaces followed by sentence:

Myself xyz

Please, enter some whitespaces followed by a sentence:

You wrote:

Hello!!

Myself xyz

C++ Manipulator showbase

C++ **manipulator showbase()** function is used to set the showbase format flag for the str stream. It forces the base of an integral value to be output. Decimal values are output by default, octal numbers are output with a leading 0, and hexadecimal numbers are output with a leading ox.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
int x = 100; // use showbase to show number base
```

```
cout << "Printing integers preceded by their base:" << endl << showbase;
```

```
cout << x << endl; // print decimal value
```

```
cout << oct << x << endl; // print octal value
```

```
cout << hex << x << endl; // print hexadecimal value
```

```
return 0; } // end main
```

Output:

Printing integers preceded by their base:

100

0144

0x64

C++ Manipulator oct

C++ **manipulator oct** function is used to set the basefield format flag for the str stream.

When we set **basefield to oct**, then integer values inserted into the stream are expressed in octal base (i.e. radix 8). For input streams, when this flag is set then extracted values are also expected to be expressed in octal base.

```
#include <iostream>
```

```
#include <iomanip>
```

```
using namespace std;
```

```
int main()
```

```
{    int n = 10000;
```

```
    cout << oct << n << endl;
```

```
    return 0;
```

```
}
```

Output:

23420

C++ Manipulator setfill

C++ manipulator setfill() function is used to specify a different character to fill the unused filled width of the value.

```
#include <iostream>
```

```
#include <iomanip>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    cout << "default fill: " << setw(10) << 42 << '\n'
```

```
        << "setfill('*'): " << setfill('*')
```

```
            << setw(10) << 42 << '\n';
```

```
}
```

Output:

default fill: 42

setfill('*'): ***42**